H01 – Life Before Frameworks

A Users REST Service on the Raw Node.js HTTP Server

Bara S. Rohmani
NRP: 5025241144
Framework-Based Programming, Informatics Engineering, ITS
2 November 2026

1 Introduction

This assignment asks for a users REST service built without Express, Fastify, or any other web framework. The only web-related building block is Node.js's built-in `node:http` module. The service supports the four CRUD operations on a `users` resource, validates incoming data, and answers every request in JSON. The goal is to see, by doing it manually, what a framework normally does for us.

My implementation lives in a single file, `src/server.ts` (302 lines), written in TypeScript and run with `tsx`. It passes all 46 tests in `src/server.test.ts`, and `tsc --noEmit` reports no type errors.

2 Environment and Tools

Item	Detail
Runtime	Node.js v22.19.0 on Windows (Git Bash)
Package manager	pnpm
Language	TypeScript 5.9.2, executed directly with `tsx`
Storage	SQLite through `better-sqlite3` 12.2.0
Testing	`vitest` 3.2.4 with `supertest` 7.1.4

I chose SQLite instead of an in-memory array so that the data survives a server restart. The `better-sqlite3` driver is synchronous, which keeps the repository code simple because there is no `await` around database calls.

3 Design Overview

I split `server.ts` into small layers, each with one job. A request flows through them from top to bottom:

Layer	Responsibility
HTTP helpers	Build response headers, serialize JSON, read and parse the request body.
Error model	A custom `HttpError` class carrying a status code and a JSON body.
Validation	`parseUserInput` checks `name`, `age`, and `isAdmin`, and collects every problem.
Repository	`createUserRepository` wraps prepared SQL statements and maps database rows to `User` objects.
Router	Matches the URL against a pattern and dispatches by HTTP method.

Server	`createMyServer` creates the `http.Server` and catches every error in one place.

The router and handlers never write to the response directly. Each handler returns a plain `HttpResult` object (`{ status, body }`), and a single function, `sendJson`, turns it into a real response. This keeps the handlers easy to read and means headers are set in exactly one place.

4 Implementation Steps

4.1 Step 1: Types and the database

I started with the data model. A `User` has `id`, `name`, `age`, `isAdmin`, `createdAt`, and `updatedAt`. SQLite has no boolean type, so `isAdmin` is stored as an integer (0 or 1). That is why there are two types: `UserRow`, which mirrors the table, and `User`, which is what the API returns. The function `toUser` converts between them.

```
type UserRow = Omit<User, "isAdmin"> & { isAdmin: number };
const toUser = (row: UserRow): User => ({ ...row, isAdmin: row.isAdmin === 1 });
```

The table is created with `CREATE TABLE IF NOT EXISTS`, so the server can start on an empty folder without any manual setup.

4.2 Step 2: A central error model

Instead of writing `res.writeHead(400, ...)` in many places, I defined `HttpError` and a small set of factory functions such as `httpErrors.invalidId()` and `httpErrors.userNotFound()`. Any layer can simply `throw` one of them. The top-level `try/catch` in `createMyServer` turns it into the right JSON response. Anything that is not an `HttpError` becomes a generic 500 response, and the real error is only logged on the server, so internal details are never leaked to the client.

4.3 Step 3: Reading the request body

Node.js does not parse request bodies. The request is a stream, so the body arrives in chunks. `readBody` collects the chunks with `for await` and joins them into a string, and `readJsonBody` calls `JSON.parse` on it. It distinguishes two failure cases: an empty body gives a validation error (“Request body is required”), while a malformed body gives “Invalid JSON”. Both cases are covered by the tests.

4.4 Step 4: Validation

`parseUserInput` receives `unknown` data and returns a trusted `UserInput`. It checks the following rules:

- `name` must be a non-empty string after trimming, with at most 100 characters.
- `age` is required and must be an integer from 0 to 150.
- `isAdmin` is optional, must be a boolean if present, and defaults to `false`.

The function collects all problems into a `details` array before throwing, so the client sees every mistake in one response rather than fixing them one at a time. The name is trimmed before it is stored, which the tests verify.

4.5 Step 5: Repository

The repository prepares each SQL statement once and reuses it. All values are passed as `?` parameters and never concatenated into the SQL string, which prevents SQL injection. `create` and `update` set `createdAt` and `updatedAt` using `new Date().toISOString()`. On creation both timestamps are identical, and on update only `updatedAt` changes.

4.6 Step 6: Routing

There is no routing library, so I wrote one with a regular expression:

```
const USERS_PATH = /^\/users(?:\/([^/]*))?$/;
```

This matches `/users` (the collection) and `/users/<something>` (one item). The captured part is passed through `parseId`, which accepts only digits. Inside each of the two handlers, a `switch` on `req.method` selects the action. Unsupported methods and unknown paths throw a 404 error.

4.7 Step 7: Server and CORS

`createMyServer` creates the server with `http.createServer`. Every response carries the same CORS headers, and `OPTIONS` requests are answered immediately so that browsers can make cross-origin calls. `src/index.ts` only imports the factory and calls `listen(3000)`. Keeping the server factory separate from `listen` is what allows the tests to start the server without occupying a real port.

5 Testing

The final result is **46 out of 46 tests passing**, and `pnpm tsc --noEmit` finishes without errors. The suite covers all four CRUD operations, validation edge cases (empty, whitespace-only, and too-long names; negative, too-large, and non-numeric ages; non-boolean `isAdmin`), ID handling, CORS headers, 404 handling, and integration scenarios such as a full create-read-update-delete cycle.

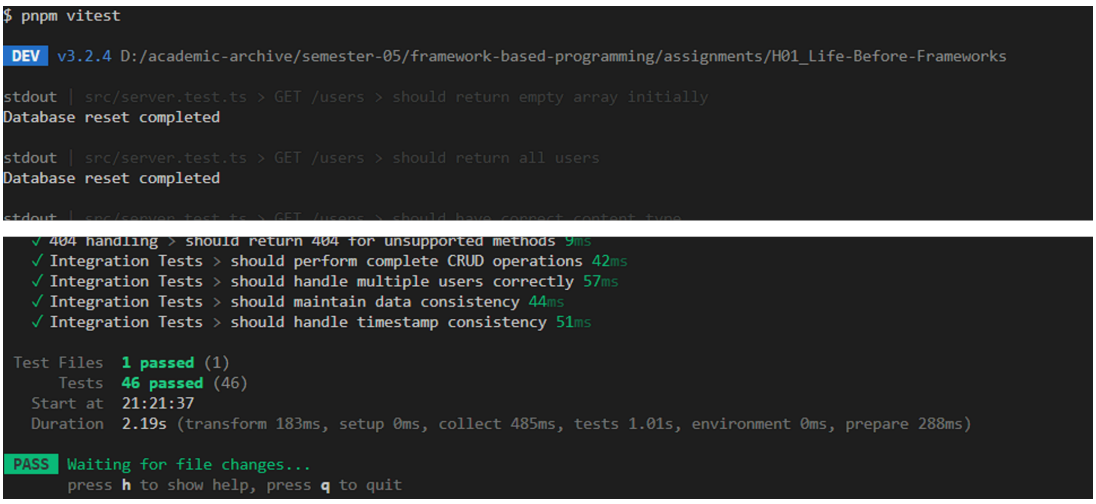


Figure 1: Result of `pnpm vitest` (individual test lines omitted).

5.1 The test that shaped the implementation most

The test that influenced my design the most is `GET /users/` **must return 400 (not 404)**. At first glance, `/users/` looks like the collection path with a trailing slash, and a simple router would either treat it as the collection or fail to match it at all and return 404. The test says it must be treated as an item request with an empty ID, which is invalid.

This forced me to use a pattern that can tell `/users` and `/users/` apart. In `USERS_PATH`, the part `(?:\/([^/]*))?` makes the whole ID segment optional, but the capture group accepts an empty string. For `/users` the captured value is `undefined`, so the request goes to the collection handler. For `/users/` it is `""`, so the request goes to the item handler, where `parseId` rejects it with 400. A pattern such as `/users/(\d+)` would have been simpler, but it would give the wrong status code. This test showed me how much a router depends on exact URL rules that I must decide myself.

5.2 A problem found through a failing test

The problem came from the tests that expect the first created user to have `id: 1`. Because the tests share one database, `resetMyServerData` has to clear the table before every test. My first idea was just `DELETE FROM users`, but then the next test created a user with `id: 2` or higher, and assertions such as `toMatchObject({ id: 1 })` failed.

The cause is the `AUTOINCREMENT` keyword in my table definition. SQLite keeps the last used ID for such a table in an internal table called `sqlite_sequence`, and deleting rows does not touch it. The fix was to also remove the `users` row from that table:

```
db.prepare("DELETE FROM users").run();
db.prepare("DELETE FROM sqlite_sequence WHERE name = 'users'").run();
```

6 Manual Testing

Besides the automated tests, I started the server with `pnpm tsx src/index.ts` and sent requests with `curl`. The figures below show two examples, one success case and one error case. The CORS headers are omitted to keep them short, but they were present in every response.

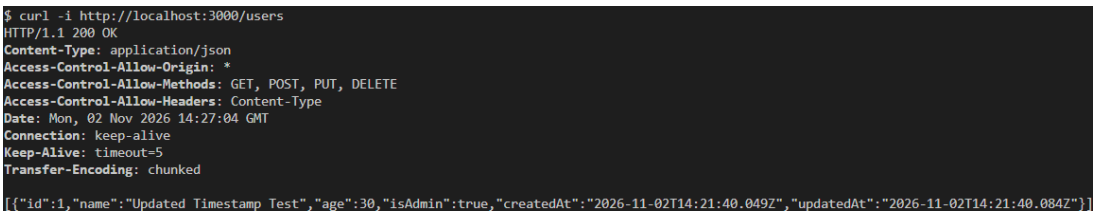


Figure 2: GET /users returns the list of users with HTTP 200 OK.

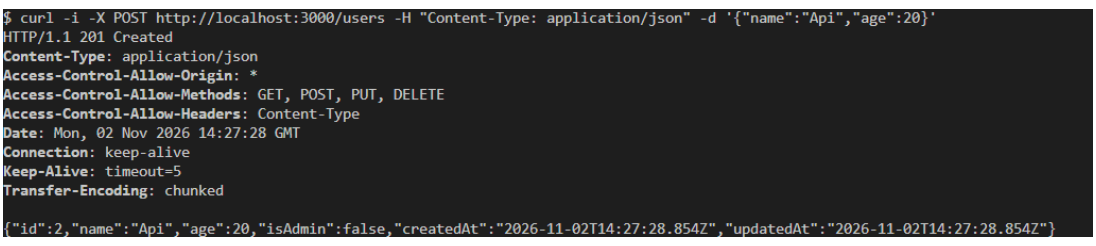


Figure 3: POST /users creates a new user and returns HTTP 201 Created.

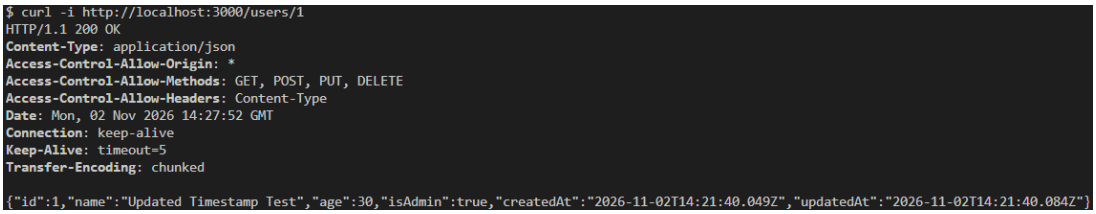


Figure 4: GET /users/1 returns the user with the requested ID.

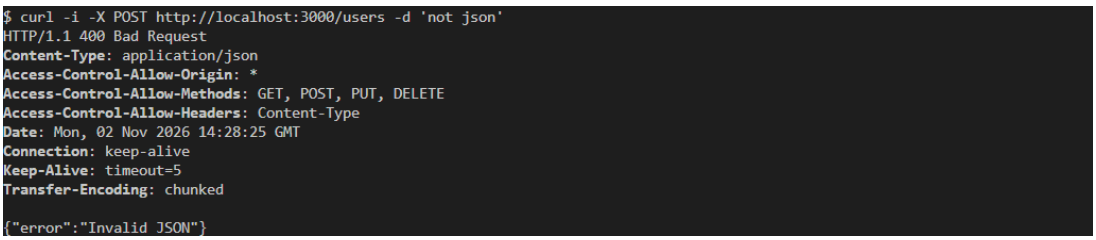


Figure 5: POST /users with malformed JSON returns HTTP 400 Bad Request.

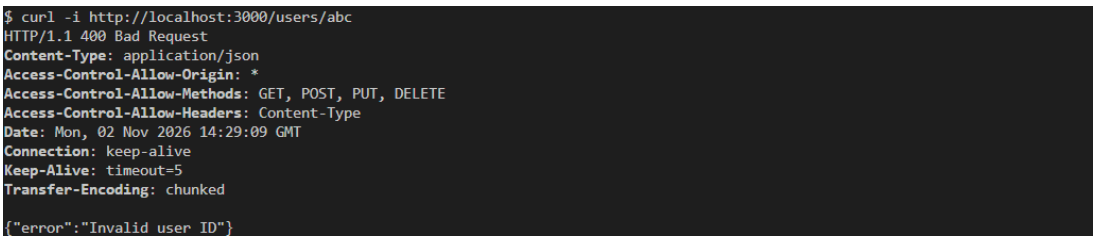


Figure 6: GET /users/abc returns HTTP 400 Bad Request because the user ID is invalid.

7 Limitations

The service meets the assignment requirements, but it is not production-ready. These are the limitations I am aware of:

- **Tests share the real database file.** The tests write to `users.db` in the working directory, the same file the real server uses, which is why leftover test data appeared in my first `curl` call. Running the tests next to real data would erase it. A separate in-memory database for tests would solve this.
- **No limit on request body size.** `readBody` keeps reading until the stream ends, so a client could send a very large body and exhaust memory.
- **No authentication or authorization.** Anyone can create, change, or delete any user, and a client can even set `isAdmin: true` for itself.
- **Single database file without migrations.** The schema is created with `CREATE TABLE IF NOT EXISTS` only. If the table structure changes later, there is no mechanism to upgrade an existing `users.db`.
- **No pagination.** `GET /users` always returns every row, which becomes slow and large as the table grows.
- **`PUT` replaces all fields.** It is not a partial update. If a client leaves out `isAdmin`, the value silently resets to `false`, which can demote an admin by accident. `PATCH` is not supported and returns 404.
- **No `Content-Type` check.** The server tries to parse any body as JSON, regardless of the declared type.
- **Database connection management.** A single global connection is created inside `createMyServer`, so calling the factory twice leaves the first connection open.

8 What I Wrote by Hand: Preparing for the Comparison

This section lists the work that a framework would normally do for me. I will use it as the baseline when comparing with the NestJS version in the next assignment.

Task in my raw implementation	Where it lives
Matching URLs and extracting the ID with a regular expression	`USERS_PATH`, `createRouter`
Dispatching by HTTP method with `switch`	`handleCollection`, `handleItem`
Collecting the request stream and parsing JSON	`readBody`, `readJsonBody`
Checking types and ranges of every field	`parseUserInput`
Converting thrown errors to status codes and JSON	`HttpError`, `toErrorResult`
Setting the same headers on every response	`RESPONSE_HEADERS`, `sendJson`
Wiring the pieces together by hand	`createMyServer`

What I learned from the implementation.

One of the more challenging parts was handling request bodies and validation. Unlike a framework that provides structured request objects, `node:http` gives me a readable stream. I had to collect the incoming chunks, combine them with `Buffer.concat`, check whether the body was empty, and then parse the JSON manually.

Another important part was separating HTTP errors from unexpected errors. Using `HttpError` and a single `catch` block made it possible to keep the route handlers focused on their main tasks instead of writing error responses repeatedly.

The routing and database operations also required more manual work than I initially expected. I had to match URLs, extract IDs, check HTTP methods, validate input fields, and prepare SQL statements myself. These are examples of repetitive tasks that a framework can help organize and reduce.

9 Conclusion

Building the service on the bare `node:http` module made every step of an HTTP request visible: reading the stream, parsing, validating, routing, handling errors, and writing headers. The result is a working, fully tested service of around 300 lines with no framework dependency. It is also clear how much of this code is repetitive plumbing rather than business logic. That observation is the starting point for evaluating what a framework such as NestJS provides.